Лабораторна робота №7

Розроблення Retrieval-Augmented Generation (RAG)

Опис:

Ця лабораторна робота спрямована на впровадження моделі RAG, яка поєднує генеративні можливості LLM з можливістю пошуку релевантної інформації у зовнішній базі знань. Модель LLM повинна використовувати базу знань для пошуку та надання відповідей з додатковим контекстом.

Мета роботи:

1. Реалізувати механізм Retrieval-Augmented Generation (RAG).
2. Інтеграція бази знань з моделлю великомасштабних мовних моделей (LLM).
3. Забезпечення генерації відповідей на основі релевантної інформації з бази знань.

Інструменти:

- **WebProtege**: https://webprotege.stanford.edu/#projects/list

Порядок виконання:

1. **Огляд архітектури RAG**
   - **Теоретичний огляд:** Ознайомтеся з основними концепціями Retrieval-Augmented Generation (RAG). Дослідіть, як працює взаємодія між двома основними компонентами:
     - **LLM** (мовна модель), яка відповідає за генерацію тексту.
     - **Система пошуку** (ретрівер), яка шукає релевантні фрагменти знань у базі даних для доповнення відповідей.
   - **Огляд прикладу архітектури:** Проаналізуйте архітектуру RAG, яка складається з наступних компонентів:
     - **Query Encoder:** Преобробка запиту користувача.
     - **Retriever:** Пошук релевантної інформації з бази знань.
     - **Generator (LLM):** Генерація відповіді на основі поєднання зовнішніх знань та внутрішньої логіки моделі.
   - **Програмне середовище:** Встановіть необхідні бібліотеки (наприклад, Hugging Face transformers, faiss для пошуку):

     ```
     pip install transformers faiss-cpu datasets
     ```

2. **Інтеграція з LLM**
   - **Завантаження попередньо натренованої моделі:** Використайте бібліотеку Hugging Face для завантаження мовної моделі, наприклад, BERT або GPT-3 для генерації тексту.

     ```
     from transformers import RagTokenizer, RagRetriever, RagTokenForGeneration
     tokenizer = RagTokenizer.from_pretrained("facebook/rag-token-nq")
     ```

```
retriever = RagRetriever.from_pretrained("facebook/rag-token-nq",
use_dummy_dataset=True)
model = RagTokenForGeneration.from_pretrained("facebook/rag-token-nq",
retriever=retriever)
```

- **Імплементація пошуку в базі знань:** Підключіть зовнішню базу знань або використайте FAISS для ефективного пошуку. Створіть або імпортуйте базу даних (наприклад, документи, статті), які будуть джерелом знань.
- **Пошук релевантної інформації:** Використайте RAG для пошуку фрагментів інформації на основі запитів користувача:

```
input_ids = tokenizer("What is the capital of France?",
return_tensors="pt").input_ids
outputs = model.generate(input_ids)
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
```

3. **Оптимізація процесу пошуку**
   - **Налаштування ефективності пошуку:** Оптимізуйте швидкість і якість пошуку, використовуючи наступні параметри:
     - **Кількість кандидатів для пошуку** (top-k): Встановіть кількість документів, які слід враховувати при пошуку.
     - **Фільтрація результатів:** Налаштуйте механізми фільтрації нерелевантної інформації на основі оцінки релевантності.
   - **Аналіз результатів пошуку:** Перевірте, наскільки коректно пошук повертає необхідну інформацію. Порівняйте результати з очікуваними відповідями.

4. **Тестування**
   - **Створення тестових запитів:** Сформуйте кілька типових запитів для перевірки, як система поєднує пошук у базі знань з генерацією відповідей.
   - **Запуск тестування:** Запустіть систему на тестових запитах і перевірте:
     - Чи правильно модель знаходить релевантну інформацію?
     - Чи коректно модель інтегрує знайдені дані у відповідь?
   - **Оцінка якості:** Проаналізуйте результати на основі метрик, таких як точність відповідей та швидкість пошуку.
   - **Оптимізація:** Налаштуйте параметри моделі або пошуку для підвищення точності та ефективності.

Результати роботи:

- Опишіть архітектуру RAG та як вона була впроваджена.
- Наведіть результати тестування та оптимізації, включаючи приклади запитів і отриманих відповідей.